

Compiladores e Intérpretes

Pautas Etapa 3

Analizador Semántico Parte 1

Universidad Nacional del Sur
Departamento de Ciencias e Ingeniería de la Computación
2026



Introducción - Aclaraciones

- El desarrollo de esta **etapa** del proyecto se **corresponde al Tema 4** de la materia
- Te recomendamos que **hayas mirado y tengas frescos** todos los temas allí tratados **antes de arrancar** el diseño/desarrollo de esta etapa
- También te recomendamos mirar el documento de **MiniJava-Semantica parte 1**
- En los ejemplos de los diseños de esta presentación vas a ver que hay "...", eso quiere decir que es muy posible que a el diseño tenga algunas/varias cosas mas

Introducción

- En esta **etapa** hay que **diseñar e implementar** un **el Chequeo de Declaraciones del Analizador Semántico** para **MiniJava**
- Para esto habrá que extender al Analizador Sintactico de la etapa anterior para que cree la Tabla de Símbolos (TS)
- El Chequeo de Declaraciones se realizará directamente sobre la TS
- Además, de manera similar a la etapa anterior va ser necesario un **Modulo Principal** que:
 - Manejará la **interfaz** con el **usuario** y
 - **Reportará los errores** detectados por el Semántico, el Sintáctico y el Léxico
 - En caso de no haber errores indicará que el análisis fue exitoso

Diseño de la TS y sus chequeos

Diseño de la Estructura de la Tabla de Símbolos

- Para diseñar la TS te recomendamos mirar el tema 4.3
- Primero hay que **identificar** las **entidades declarables**
 - La idea es que cada **entidad declarable** será una **clase**
 - Luego si una **entidad** puede “**contener**” otras entidades tendrá **tablas/hashs** con esas **entidades**

TablaDeSimbolos

```
Hash<Clase> clases  
...
```

Clase

```
Hash<Atributo> atributos  
Hash<Metodo> metodos  
...
```

Metodo

```
Tipo TipoRetorno  
Hash<Parametro> params  
...
```

Diseño de la Construcción de la Tabla de Símbolos

- La **TS** se **construye** a medida que se realiza el **análisis sintáctico**
 - Para **diseñar** que hacer usamos una **EDT**
 - Esta EDT debe realizarse sobre la **versión de la gramática** de MiniJava que usaron para **implementar el analizador sintáctico**
 - La idea es que las **acciones** de la EDT vayan **construyendo la TS** a medida que vamos **reconociendo sintacticamente** cada declaración de una entidad
- **Importante:** la TS debe ser global en la EDT!

Diseño del Chequeo de Declaraciones

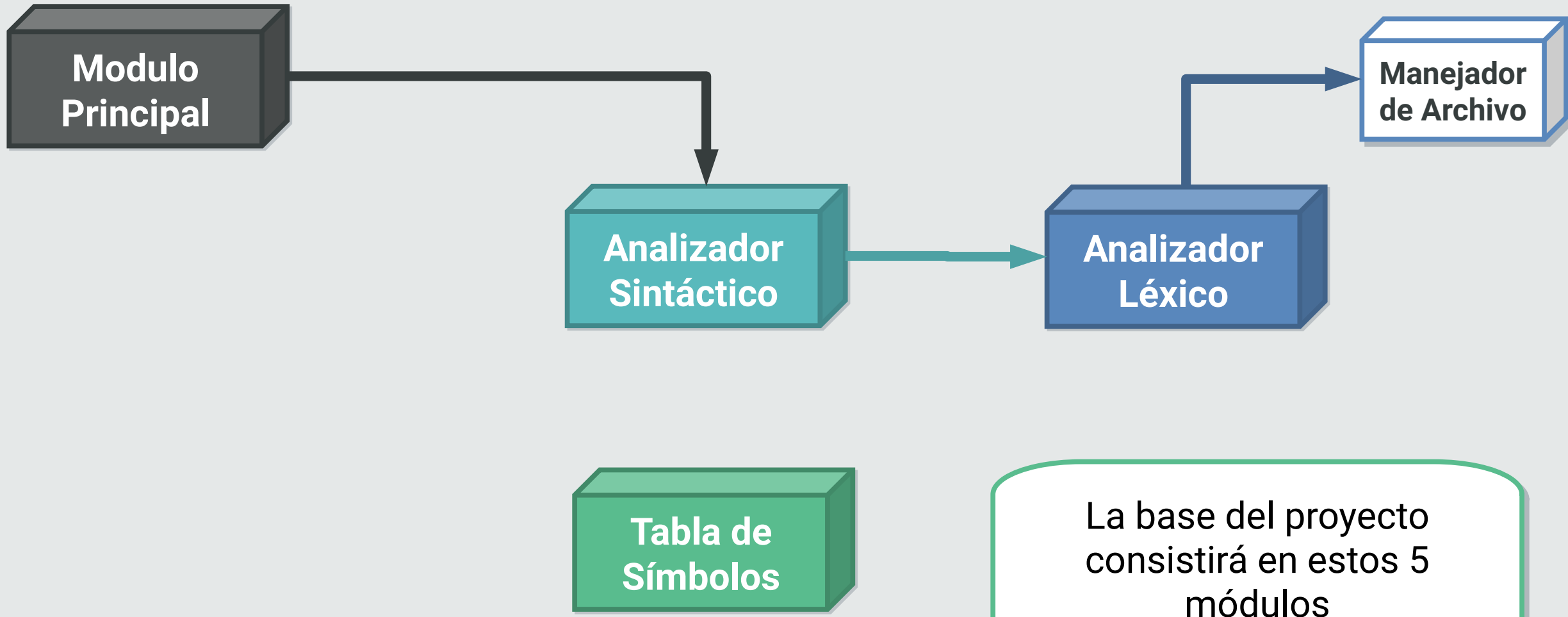
- Para el Chequeo de Declaraciones te recomendamos mirar el tema **la Clase 8**
- Consiste de **dos tareas** principales
 - Control de la **correctitud de declaración** de todos los **elementos** de la **TS**
 - **Consolidación** de la **TS**
- Estas tareas se pueden diseñar como funciones/métodos directamente sobre la TS y las clases de las Entidades

Diseño del Chequeo de Declaraciones

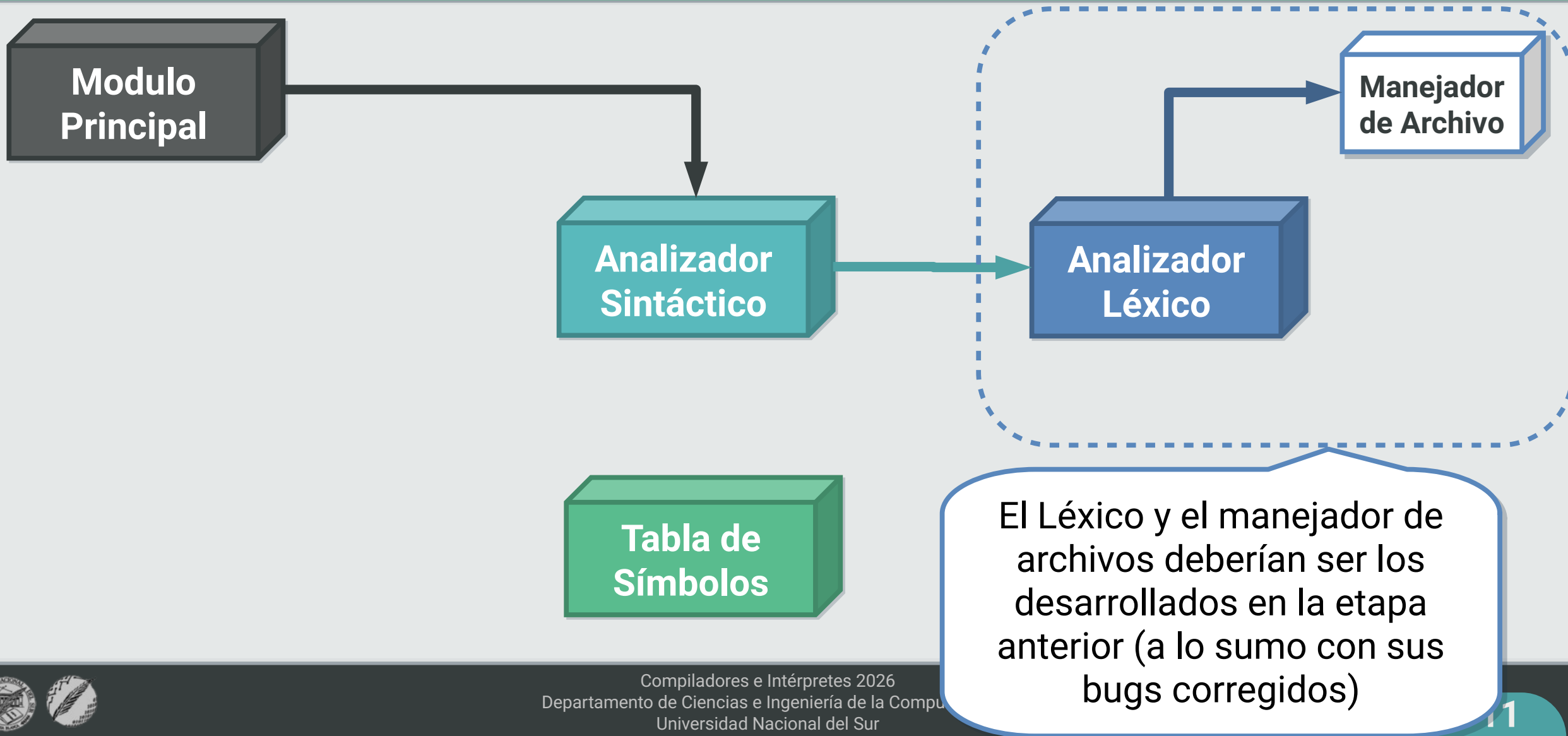
- Para diseñar esta tarea la clave es valernos de la **Delegación** y las **Responsabilidades** de cada entidad
- Por Ejemplo, para el control de correctitud
 - La idea es que cada entidad **controla** lo que le **corresponde** (su **responsabilidad**) y **solicita** a cada entidad que **contiene** que se controle (**delega** el control)

Estructura y Funcionalidad del Software a Desarrollar

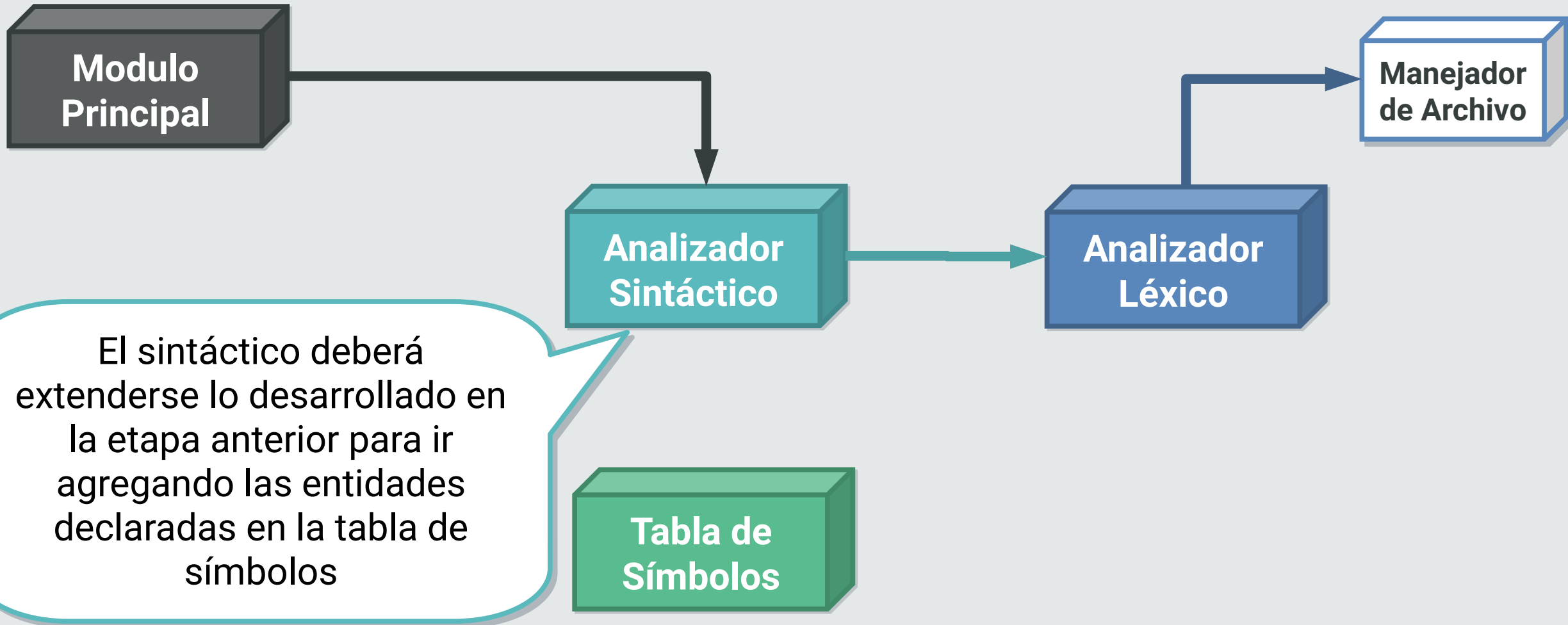
Esquema General



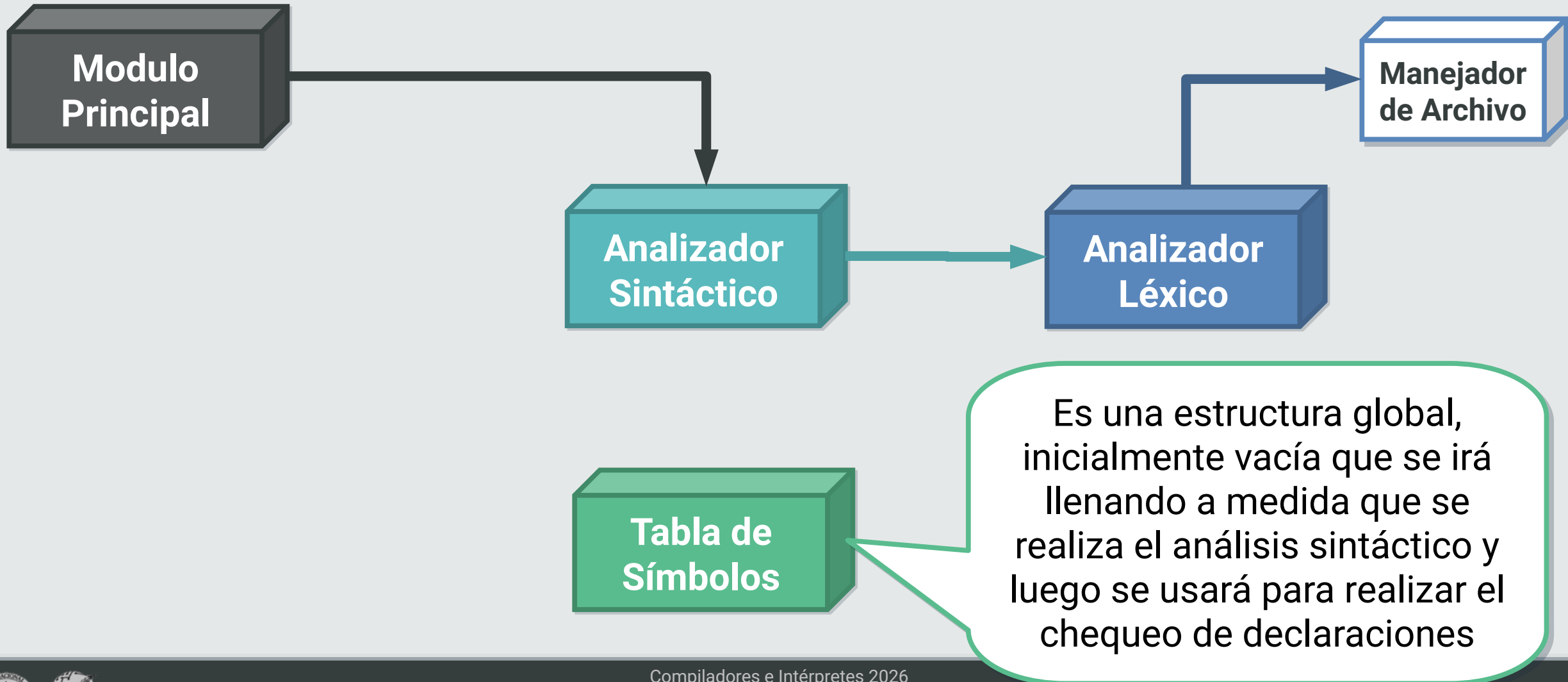
Esquema General



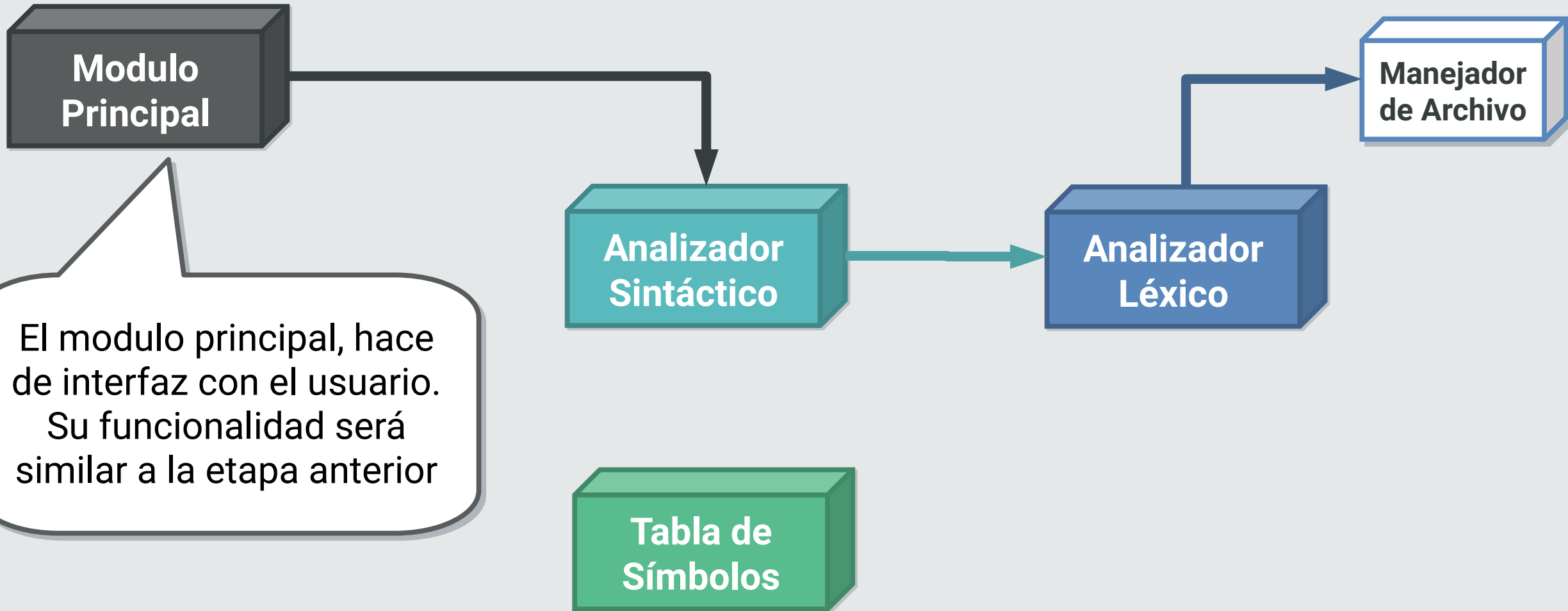
Esquema General



Esquema General



Esquema General



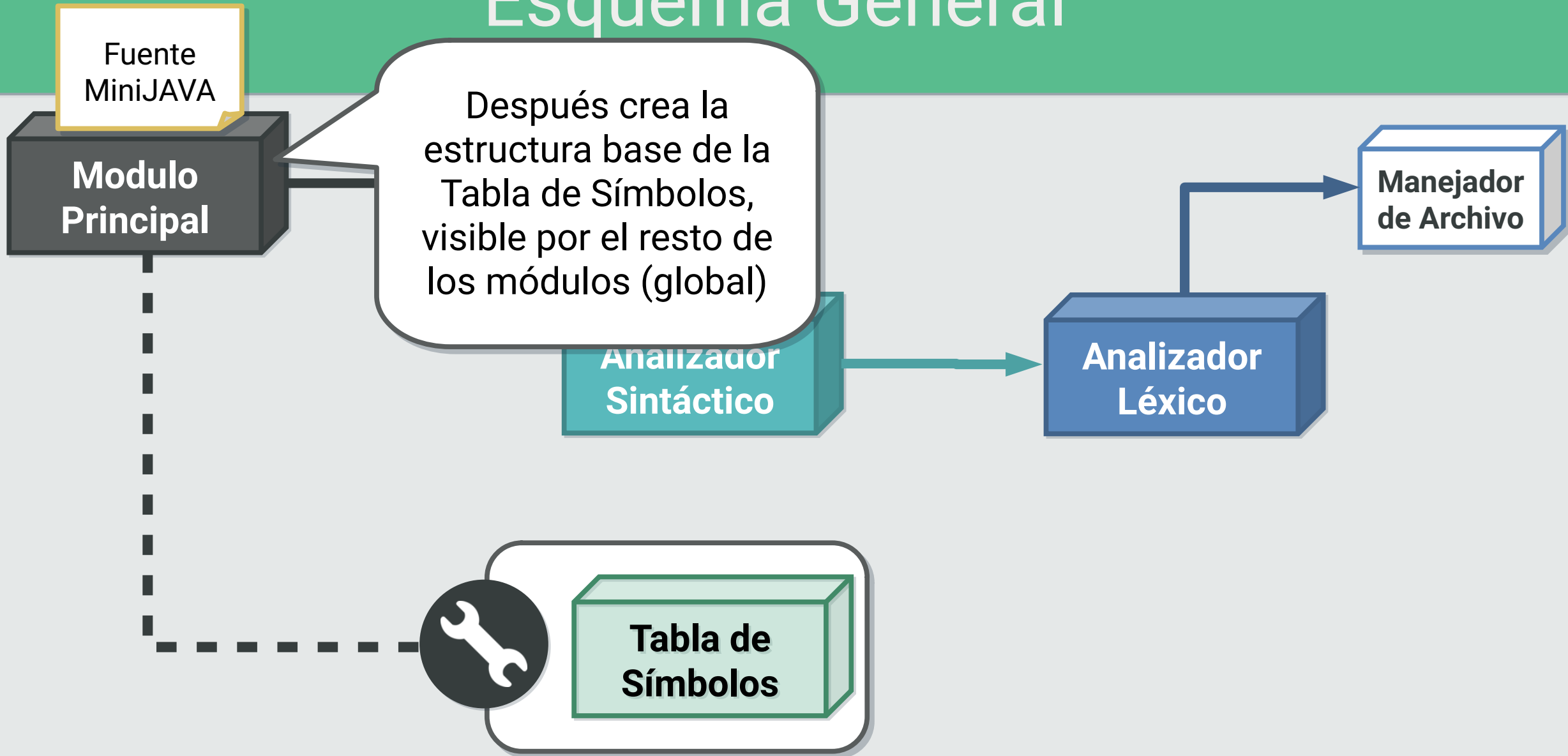
Esquema General



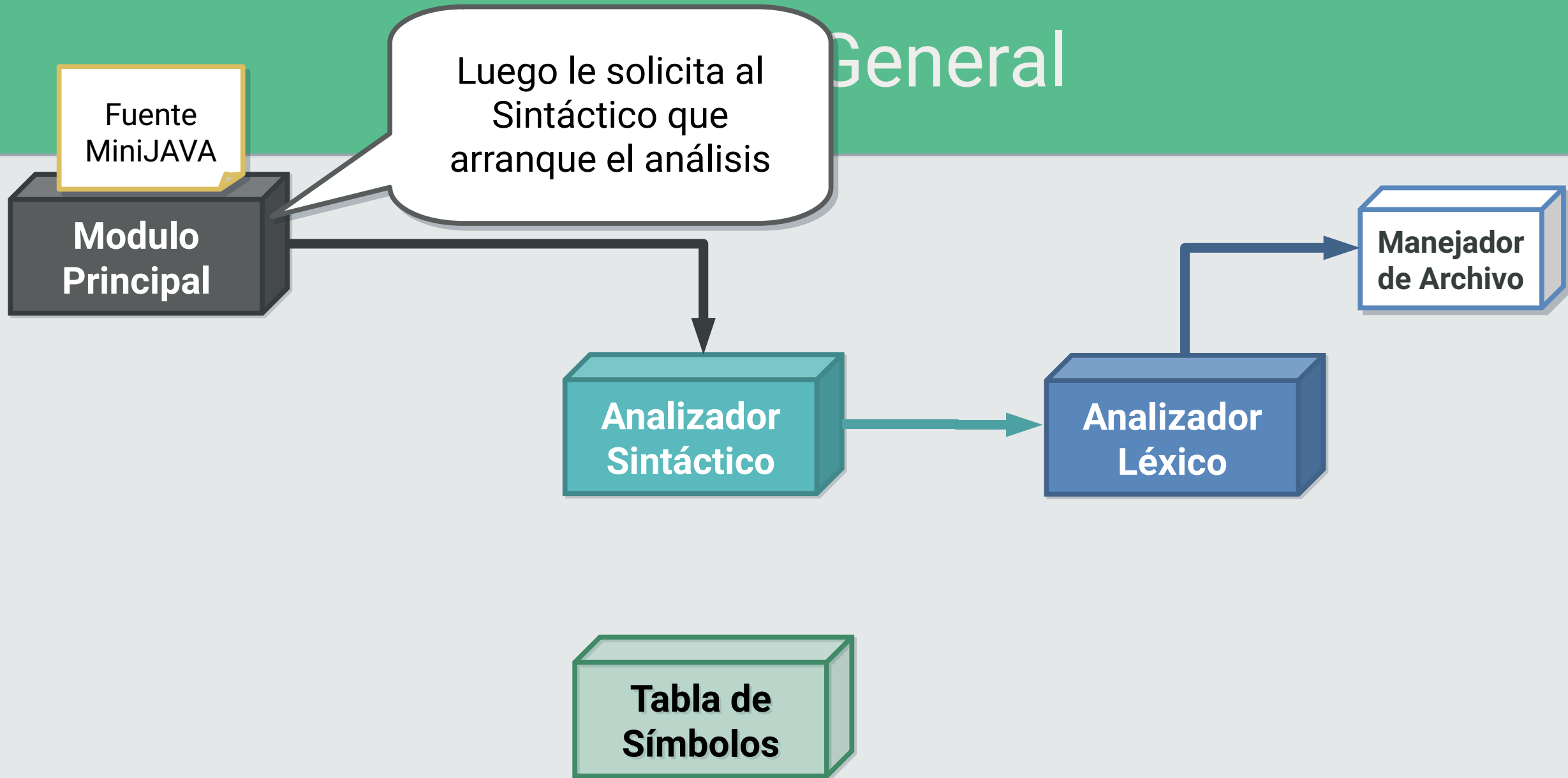
Esquema General



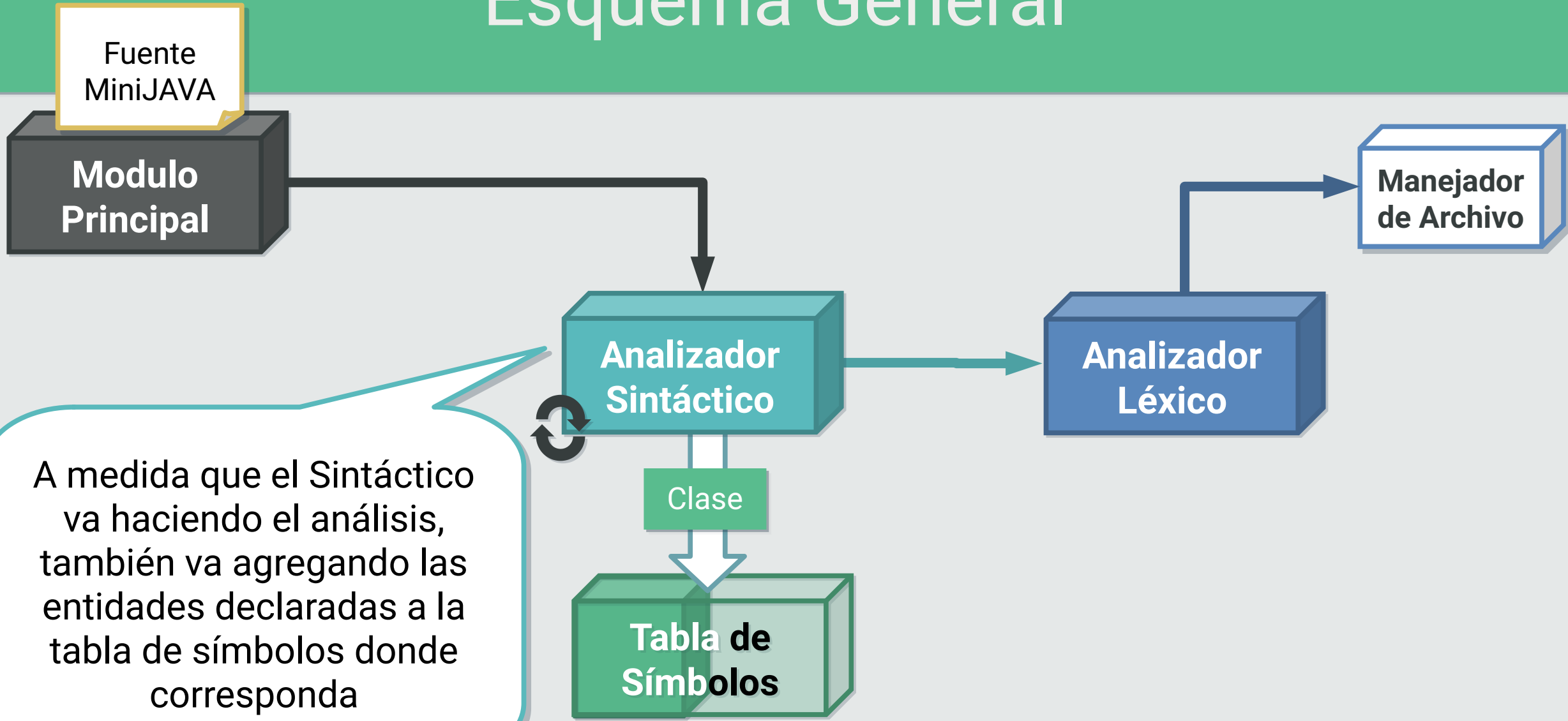
Esquema General

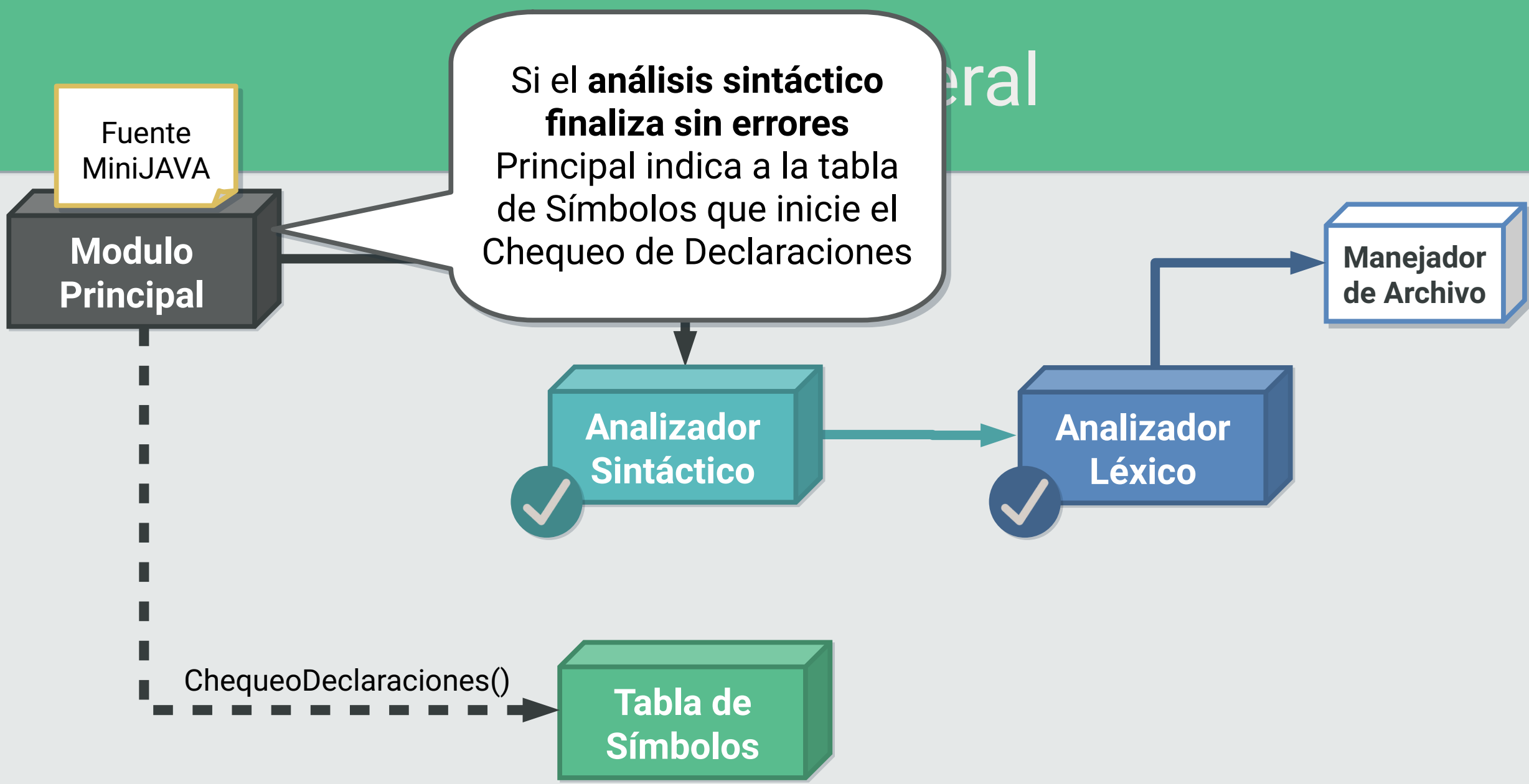


General

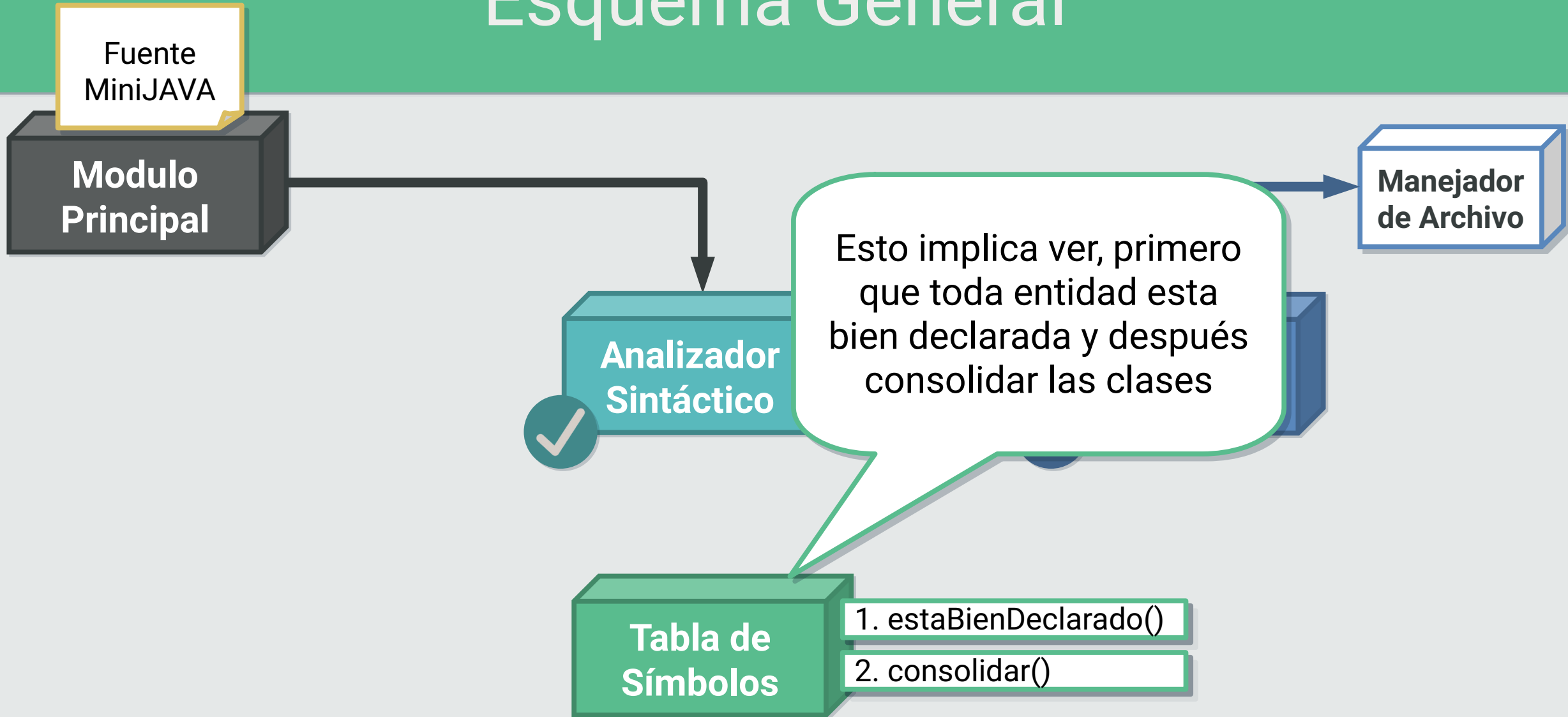


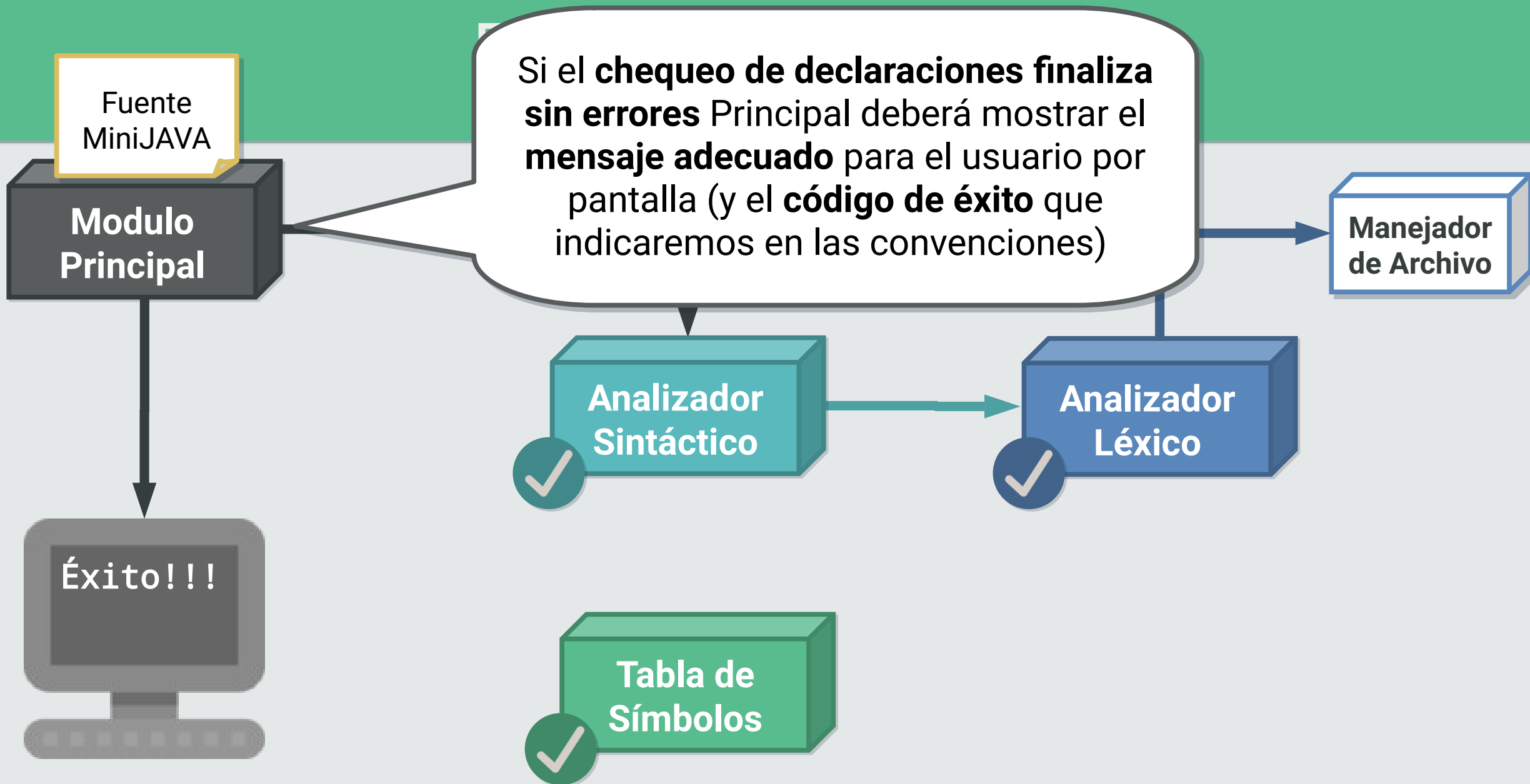
Esquema General



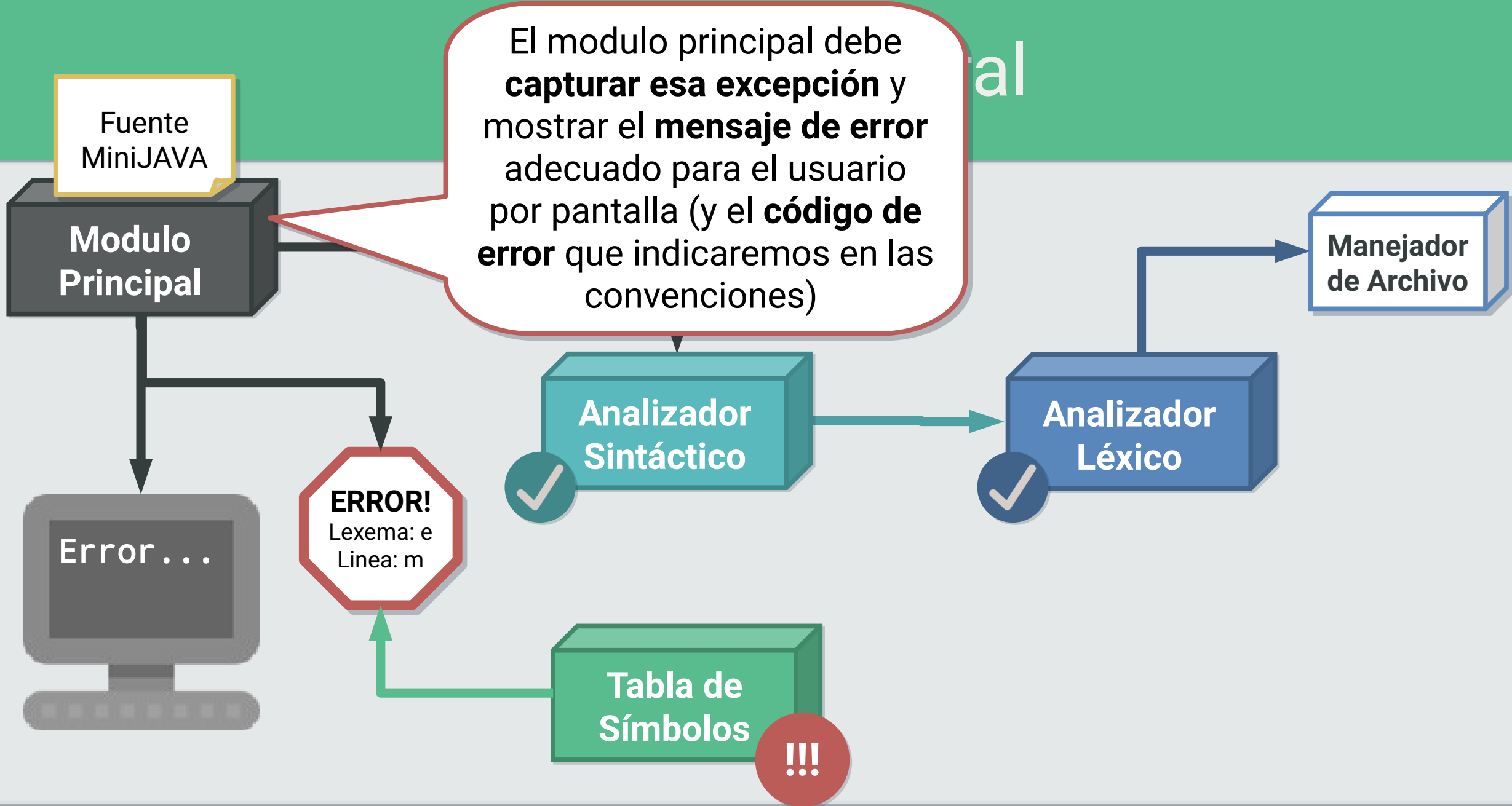


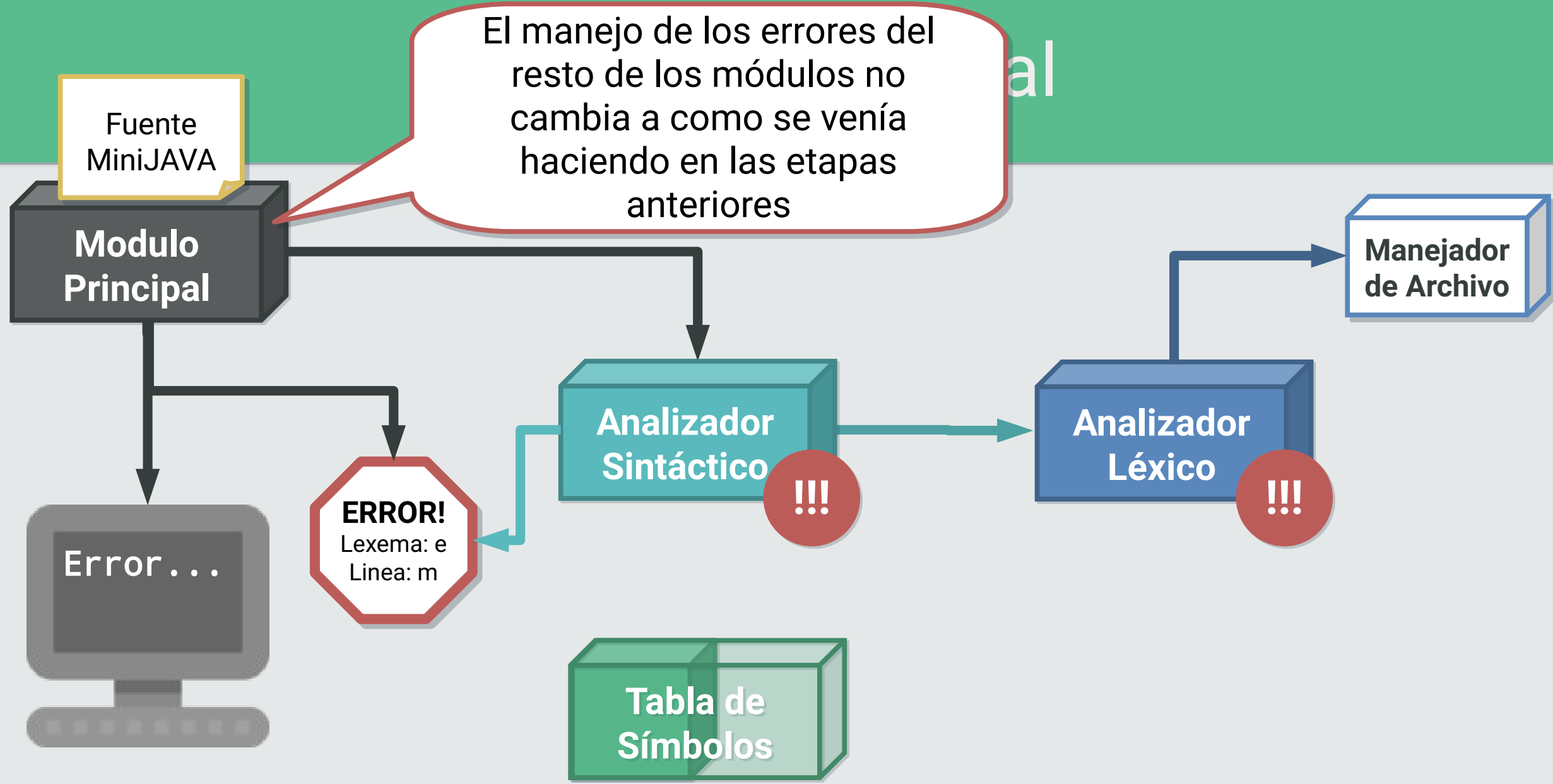
Esquema General











Convenciones y Consideraciones

Convenciones y Consideraciones

Fecha Limite de Entrega: 09/10/2026 a las 16hs

Forma de Entrega: En la actividad asociada a la entrega etapa 3 en Moodle, subiendo un zip con todo lo requerido

Convenciones y Consideraciones

Reportes de Errores

- Al detectar un error deberá reportarse y finalizar la ejecución
- Deberá reportarse un mensaje adecuado a la naturaleza del error indicando el lexema del token que produce el error y en que numero de línea se encuentra.
- Al final del reporte deberá mostrarte un código de error con la siguiente estructura **[Error:Lexema|NroLinea]** donde Lexema y Nro de Linea son los reportados en el mensaje arriba mencionado.
- Ejemplo:



Fuente MiniJava

```
class A extends B{  
}
```

El Reporte de
Error debería ser

```
Error Semantico en linea 1: La clase B no  
esta declarada  
[Error:B|1]
```

Convenciones y Consideraciones

Reportes de Errores

- Al detectar un error deberá reportarse y finalizar la ejecución
- Deberá reportarse un mensaje adecuado a la naturaleza del error indicando el lexema del token que produce el error y en que numero de línea se encuentra.
- Al final del reporte deberá mostrarte un código de error con la siguiente estructura **[Error:Lexema|NroLinea]** donde Lexema y Nro de Linea son los reportados en el mensaje arriba mencionado.
- Otro Ejemplo:



Fuente MiniJava

```
class A{}  
class A{}
```

El Reporte de
Error debería ser

```
Error Semantico en linea 2: La clase A ya  
esta declarada  
[Error:A|2]
```

Convenciones y Consideraciones

Reportes de Errores

- Al detectar un error deberá reportarse y finalizar la ejecución
- Deberá reportarse un mensaje adecuado a la naturaleza del error indicando el lexema del token que produce el error y en que numero de línea se encuentra.
- Al final del reporte deberá reportarse un código de error con la



Error!

Aclaraciones:

- Cuando el problema se da por un nombre repetido, el token es el que fue declarado posteriormente desde el punto de vista sintáctico
- Cuando un **método** está **mal redefinido** el token corresponde con su nombre en la clase descendiente (es decir el que está mal redefinido)
- Cuando hay un token refiriendo a un tipo no declarado, usar el lexema y línea de ese token para reportar el error
- Cuando no hay un token claramente asociado a la causa del problema (por ejemplo un método de interface no implementado) usar el token del nombre de la entidad que tiene el problema (en el ejemplo sería de la clase descendiente)

Convenciones y Consideraciones

Mensaje de Análisis Exitoso

- Cuando no se detectan errores en el análisis el Modulo Principal debería mostrar un mensaje adecuado y luego el código de éxito: **[SinErrores]**
- Por ejemplo

Fuente MiniJava

```
class C1{  
}
```

El mensaje
debería ser

Compilacion Exitosa
[SinErrores]



Convenciones y Consideraciones



No cumplir con estas convenciones llevará a incurrir en fallos y/o a la desaprobación de la entrega/reentrega

Convenciones y Consideraciones



No cumplir con estas convenciones llevará a incurrir en fallos y/o a la desaprobación de la entrega/reentrega

Las convenciones son esenciales para que **nosotros podamos corregir** bien la entrega...
por eso somos tan estrictos!

Si no la podemos corregir
va a estar **desaprobada!!!**

Herramientas de Test para la Evaluación

- En la tarea también les adjuntamos **herramientas de testing** de JUnit
- Estas herramientas las utilizamos en la cátedra como parte de la **evaluación** de los proyectos
- En el adjunto van a encontrar una **guía** de como agregarlas al proyecto (está armada respecto a **IntelliJ IDEA**)

Aclaración Importante: Las herramientas **no incluyen** los **casos de prueba** necesarios para **verificar adecuadamente** el compilador. El desarrollo de los casos es **responsabilidad del estudiante**. *Ademas, estas no son las únicas herramientas que utiliza la cátedra para la evaluación*

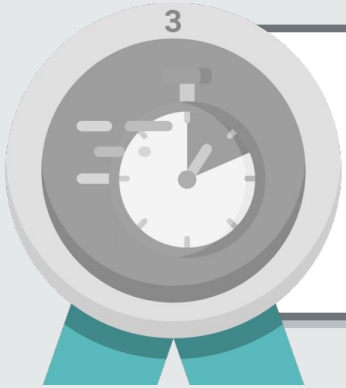
Informe

Informe

- El informe deberá indicar **como compilar el código fuente** entregado y como correrlo una vez compilado.
- Diagrama de Clases de la tabla des simbolos
- Debe indicarse que **logros** se intentan alcanzar en la Etapa

Logros Disponibles para la Etapa 3

Logros Etapa 3



Entrega Anticipada Semántica I

(no valido para la reentrega, ni etapas subsiguiente)

La entrega debe realizarse 48hs antes de la fecha limite

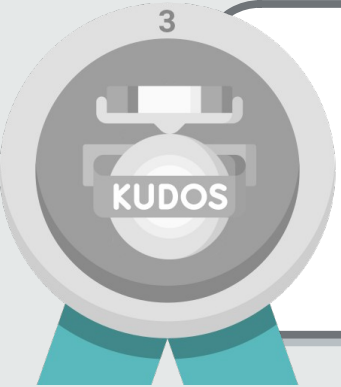


Imbatibilidad Semantica I

(no valido para la reentrega, ni etapas subsiguiente)

El software entregado pasa correctamente toda la batería de prueba utilizada por la cátedra

Logros Etapa 3



Kudos!

(no valido para la reentrega, ni etapas subsiguiente)

Gran trabajo! La entrega tiene solo algunos errores menores

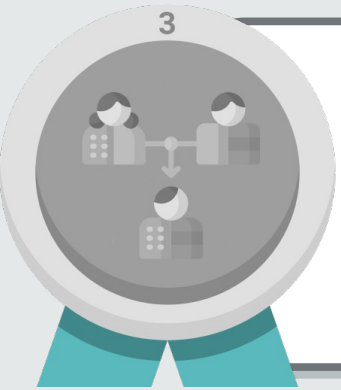


Sobrecarga! E3

El compilador permite usar los modificadores sealed y final como en Java. Seal solo aplica a clases e interfaces, mientras que final aplica a clases, interfaces y métodos.



Logros Etapa 3



Herencia Multiple E3

El compilador permite herencia multiple de interfaces como en Java



Métodos Genericos E3

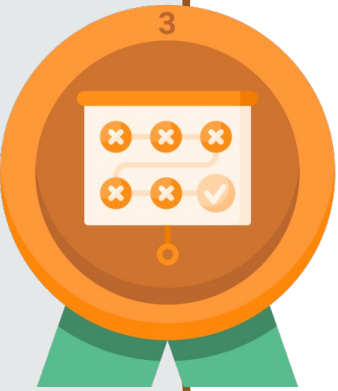
El compilador permite declarar métodos genericos como Java. Las reglas de redefinicion para estos métodos siguen las mismas que en Java.



Logros Etapa 3

Multi-Detección Errores Semanticos E3

El compilador no finaliza la ejecución ante el primer error semantico en una declaración (se recupera) y es capaz de reportar otros errores que tenga el programa fuente en una corrida. Como mínimo se espera que el compilador pueda continuar con el análisis de la próxima declaración y que cuando hay nombres repetidos descarte ambas entidades.



Genericidad Avanzada E3

El compilador controla que los tipos genericos anidados son correctamente utilizados en declaraciones

Requiere:



Logros de Etapas Anteriores

- Recordar que es posible también intentar alcanzar **logros** de las **etapas anteriores**
- Aun así chequear que el logro de sea alcanzable en una etapa posterior ya que existen algunos que no lo permiten (como por ejemplo entrega anticipada o imbatibilidad)